



المعهد الوطني للبريد والمواصلات
ⵎⵓⵙⵙⵓⵔⵉⵏ ⵙⵓⵎⵓⵏⵉ ⵙⵓⵎⵓⵏⵉ ⵙⵓⵎⵓⵏⵉ
Institut National des Postes et Télécommunications

INSITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

SMART-ICT

Kogito Keployment

Realisé par :
AGOURAM YASSINE

Encadrant :

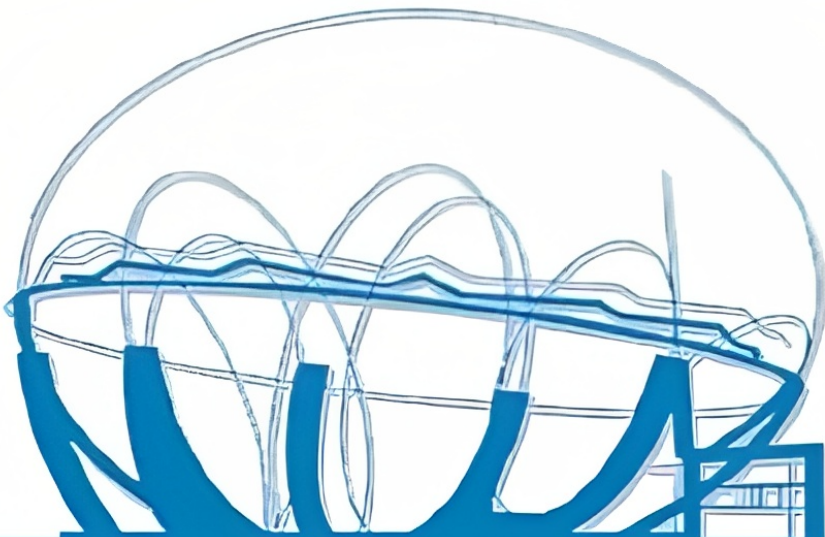


Table des matières

Introduction générale	2
1 Objectifs de l'EPIC	2
2 Cycle de vie d'une application Kogito :	2
2.1 Phase de Design	2
2.2 Phase de Build	3
2.3 Phase de Deploy	3
2.4 Conclusion	3
3 Cycle de déploiement d'une application Kogito	3
3.1 Vue d'ensemble du flux	4
3.2 Description des étapes	4
3.2.1 Kogito Project	4
3.2.2 Construction de l'image	4
3.2.3 Kubernetes Custom Resource	4
3.2.4 Déploiement du service Kogito	4
3.3 Interaction entre les composants	5
4 Kogito Operator	5
5 Anatomie d'un service Kogito	6
5.1 Vue d'ensemble	6
5.2 Construction et stockage de l'image	7
5.3 Déploiement des instances (Pods)	7
5.4 Gestion de la configuration (ConfigMaps)	7
5.5 Exposition interne via Service Kubernetes	7
5.6 Exposition externe via Ingress	7
5.7 Rôle global de l'Operator (implicite)	8
5.8 Conclusion	8
6 Architecture globale du système Kogito	8
6.1 Interaction utilisateur et point d'entrée	8
6.2 Exposition via Ingress et orchestration Kubernetes	9
6.3 Services de processus Kogito	9
6.4 Communication événementielle avec Kafka	9
6.5 Services consommateurs et traitement asynchrone	9
6.6 Persistance des données	10
6.7 Supervision et observabilité	10
6.8 Conclusion	10
7 Mode de déploiement	10
Conclusion générale	12

Introduction générale

Dans le cadre de la migration vers une architecture cloud-native basée sur Kubernetes et Kogito, cet EPIC vise à établir une fondation robuste, sécurisée et scalable. L'objectif principal est de définir une architecture cible cohérente, tout en mettant en place les mécanismes essentiels de sécurité, d'observabilité et d'intégration middleware.

Cette phase est critique, car elle conditionne la stabilité, la maintenabilité et l'évolutivité de l'ensemble du système.

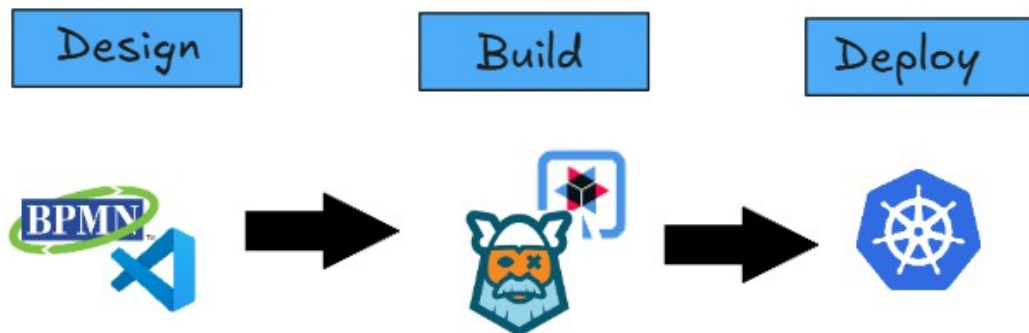
1 Objectifs de l'EPIC

Les objectifs principaux de cet EPIC sont :

- Définir une architecture cible Kogito adaptée à Kubernetes
- Structurer le cluster via des namespaces logiques
- Mettre en place un socle de sécurité complet
- Intégrer les composants middleware nécessaires (messaging, stockage)
- Adopter une approche GitOps pour le déploiement

2 Cycle de vie d'une application Kogito :

Ce schéma représente le cycle de vie d'une application Kogito, structuré en trois phases principales : Design, Build et Deploy, illustrant une chaîne complète de développement et de déploiement dans un environnement cloud-native.



2.1 Phase de Design

La première étape correspond à la conception des processus métier. Les développeurs modélisent les workflows en utilisant le standard BPMN, qui permet de représenter graphiquement les processus (tâches, événements, décisions, etc.).

Des outils de développement comme Visual Studio Code peuvent être utilisés pour créer et éditer ces fichiers BPMN, souvent avec des extensions dédiées à Kogito. Cette phase vise à définir clairement la logique métier de manière visuelle et compréhensible.

2.2 Phase de Build

Une fois les processus conçus, ils passent à la phase de build (construction). Ici, le moteur Kogito entre en jeu.

Kogito génère automatiquement :

Du code exécutable à partir des fichiers BPMN Des services prêts à être déployés (souvent sous forme de microservices)

Cette étape transforme donc les modèles métier en applications concrètes, prêtes à fonctionner dans un environnement cloud.

2.3 Phase de Deploy

Enfin, l'application est déployée dans un cluster Kubernetes.

Kubernetes prend en charge :

- L'orchestration des conteneurs
- La scalabilité automatique
- La haute disponibilité
- La gestion du cycle de vie des services

Les applications Kogito, une fois construites, sont empaquetées (généralement sous forme de conteneurs Docker) puis déployées sur Kubernetes pour être exécutées à grande échelle.

2.4 Conclusion

Ce pipeline Design → Build → Deploy illustre une approche moderne du développement :

Design : modélisation métier (BPMN)

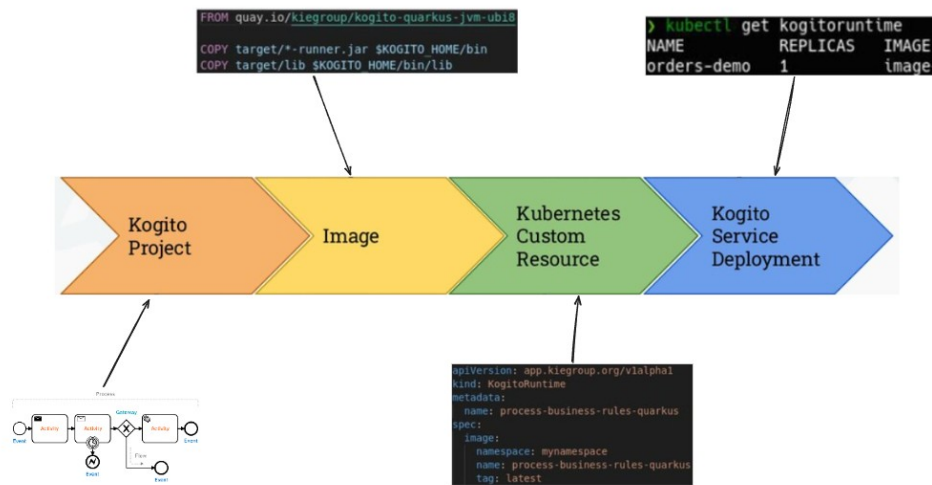
Build : génération et construction avec Kogito

Deploy : exécution dans Kubernetes

Ce modèle permet de passer rapidement d'une idée métier à une application déployée, tout en assurant une forte automatisation et une bonne intégration avec les environnements cloud-native.

3 Cycle de déploiement d'une application Kogito

L'image illustre le cycle de déploiement d'une application basée sur Kogito, depuis la conception du projet jusqu'à son exécution dans un environnement Kubernetes. Elle met en évidence les différentes étapes techniques et les artefacts associés à chaque phase.



3.1 Vue d'ensemble du flux

Le schéma présente une chaîne linéaire composée de quatre blocs principaux : Kogito Project → Image → Kubernetes Custom Resource → Kogito Service Deployment

Chaque étape transforme ou enrichit l'application pour la rendre exécutable dans un cluster.

3.2 Description des étapes

3.2.1 Kogito Project

Cette première étape correspond à la phase de développement.

Le projet inclut des modèles métier (ex. BPMN pour les processus). Un exemple visuel en bas à gauche montre un diagramme de workflow (activités, événements, passerelles). Le code source et les définitions métier sont préparés pour la compilation.

3.2.2 Construction de l'image

À partir du projet, une image conteneur est générée.

Le snippet en haut montre un extrait de Dockerfile utilisant une image de base (kogito-quarkus). Les fichiers compilés (.jar, bibliothèques) sont copiés dans l'image. Cette image constitue une unité déployable standard compatible avec les environnements cloud.

3.2.3 Kubernetes Custom Resource

L'image est ensuite référencée dans une ressource personnalisée Kubernetes (CRD).

Le bloc YAML en bas montre une ressource de type KogitoRuntime. Cette configuration inclut : le nom du service le namespace l'image à déployer Cette abstraction simplifie le déploiement en encapsulant les paramètres nécessaires.

3.2.4 Déploiement du service Kogito

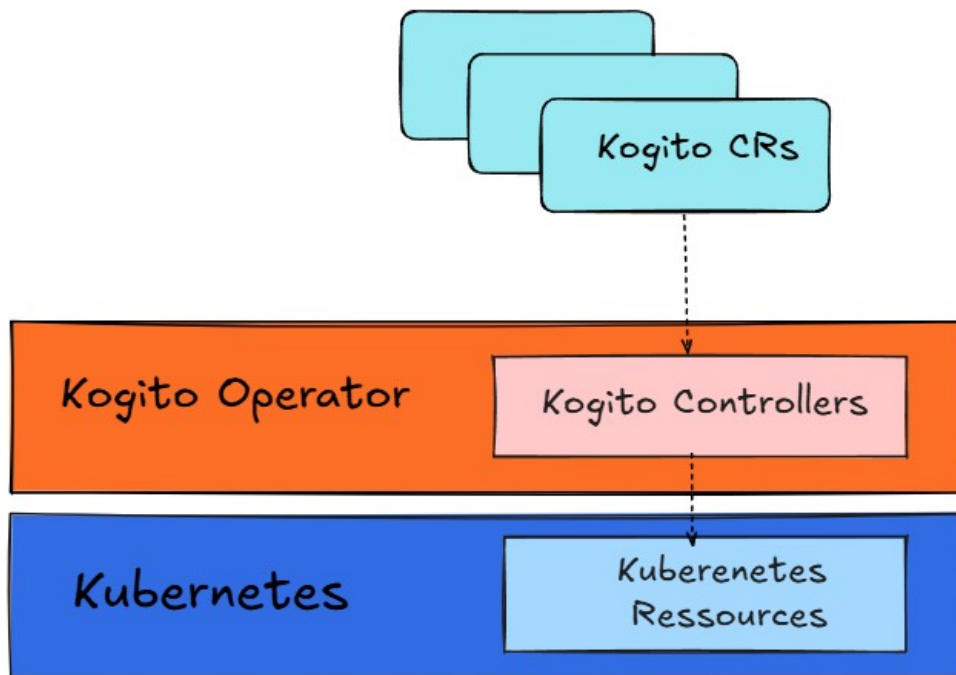
La dernière étape correspond à l'exécution dans Kubernetes.

La commande `kubectl get kogitoruntime` (en haut à droite) permet de vérifier l'état du déploiement. Le service est lancé avec un certain nombre de répliques (ici 1). L'application devient accessible et opérationnelle dans le cluster.

3.3 Interaction entre les composants

Le projet Kogito fournit la logique métier. L'image conteneur encapsule cette logique avec son environnement d'exécution. La ressource Kubernetes décrit comment déployer cette image. Kubernetes orchestre le déploiement et assure la disponibilité du service.

4 Kogito Operator



Au niveau supérieur, on trouve les Kogito CRs (Custom Resources). Il s'agit de ressources personnalisées définies par l'utilisateur pour décrire les applications Kogito (processus métier, services, configurations, etc.). Ces CRs permettent d'exprimer l'état désiré du système de manière déclarative.

Ensuite, le Kogito Operator agit comme un composant intermédiaire intelligent. Il surveille en continu les Kogito CRs et interprète leur contenu. À l'intérieur de cet opérateur, les Kogito Controllers sont responsables de la logique de traitement : ils traduisent les définitions des CRs en actions concrètes. Autrement dit, ils prennent les spécifications déclarées et les convertissent en ressources exploitables par Kubernetes.

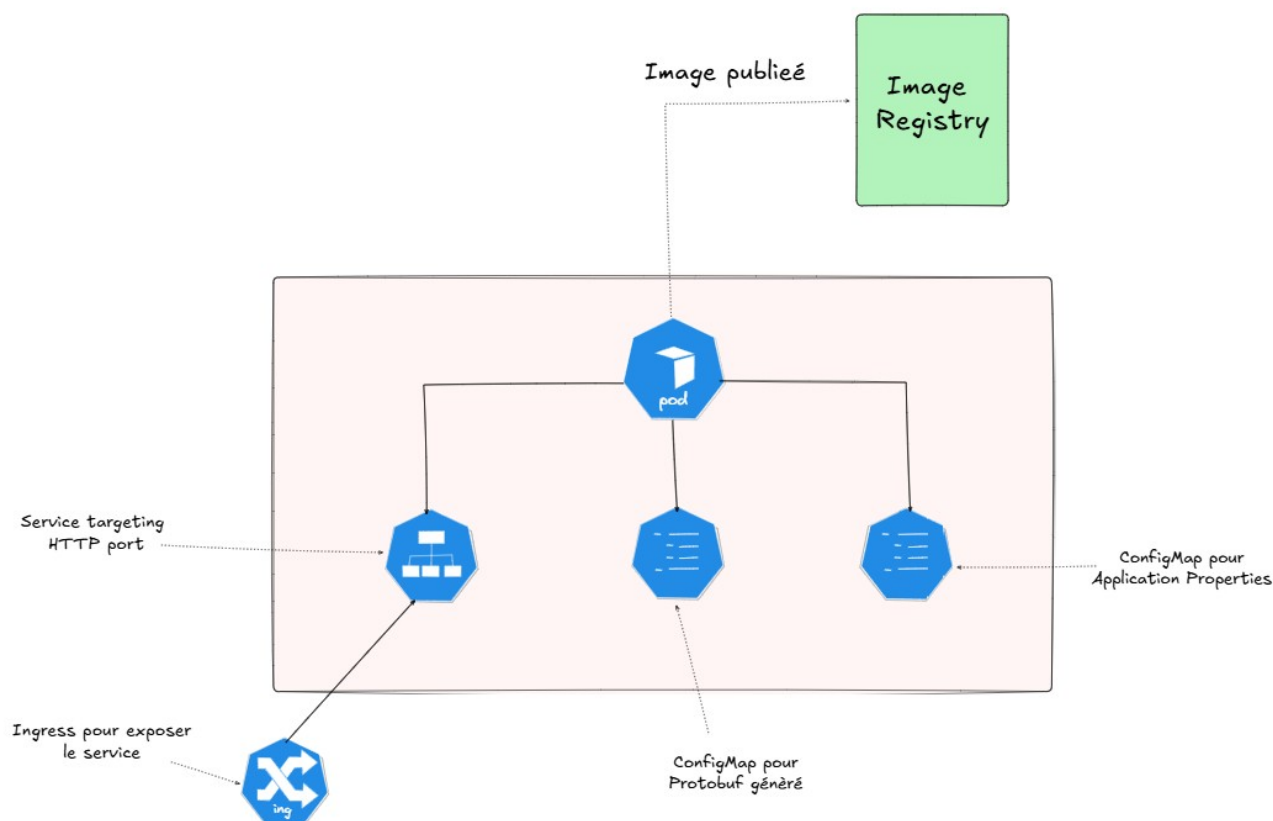
Enfin, au niveau inférieur, se trouve Kubernetes, qui exécute réellement les applications. Les Kubernetes Ressources (pods, services, deployments, etc.) sont automatiquement

créées et gérées par l'operator en fonction des CRs. Kubernetes assure alors l'orchestration, la scalabilité, la résilience et la gestion du cycle de vie des applications.

En résumé, ce modèle suit une approche déclarative : l'utilisateur définit ce qu'il souhaite via les Kogito CRs, le Kogito Operator se charge de la traduction et de la gestion, et Kubernetes exécute l'infrastructure nécessaire. Cela simplifie fortement le déploiement et l'exploitation des applications cloud-native basées sur Kogito.

5 Anatomie d'un service Kogito

Le schéma présente l'architecture de déploiement et d'exposition d'un service basé sur Kogito, une technologie orientée microservices pour l'exécution de processus métier et de règles, généralement déployée sur Kubernetes ou OpenShift.



5.1 Vue d'ensemble

L'architecture illustre le cycle de vie d'un service Kogito, depuis la construction de l'image jusqu'à son exposition externe. Elle met en évidence les interactions entre plusieurs composants clés :

Un registre d'images (Image Registry) contenant les images applicatives
Des pods Kubernetes exécutant le service
Des ConfigMaps pour la configuration dynamique
Un Service Kubernetes pour l'accès interne
Un Ingress pour l'exposition externe

5.2 Construction et stockage de l'image

Le processus débute par la création d'une image applicative Kogito :

L'image est construite à partir du code source (processus métier, règles, API REST). Une fois générée, elle est publiée dans un registre d'images. Cette image constitue l'unité déployable utilisée par Kubernetes pour instancier les conteneurs.

5.3 Déploiement des instances (Pods)

Dans la zone centrale (encadrée), on observe plusieurs instances du service :

Chaque instance est représentée par un pod Kubernetes. Ces pods exécutent des conteneurs basés sur l'image précédemment publiée. Le déploiement est généralement géré par un contrôleur (Deployment ou Operator Kogito).

Le composant supérieur (représenté par un cube) agit comme orchestrateur du déploiement :

- Il récupère l'image depuis le registre

- Il instancie plusieurs pods pour assurer la scalabilité et la haute disponibilité

5.4 Gestion de la configuration (ConfigMaps)

Deux types de ConfigMaps sont utilisés :

- a. ConfigMap pour les propriétés applicatives

Contient les paramètres de configuration (variables d'environnement, endpoints, etc.) Permet de modifier le comportement de l'application sans reconstruire l'image

- b. ConfigMap pour le protobuf généré

Contient les définitions de données (schémas protobuf) Utilisé notamment pour la sérialisation et la communication avec des systèmes externes (ex : Kafka, Infinispan)

Ces ConfigMaps sont montées dans les pods ou injectées comme variables d'environnement.

5.5 Exposition interne via Service Kubernetes

Un Service Kubernetes est configuré pour :

- Fournir un point d'accès stable aux pods
- Effectuer un load balancing entre les différentes instances
- Cibler un port HTTP spécifique (mentionné comme "Service targeting HTTP port")

Ce service abstrait la dynamique des pods (création/suppression) et garantit la continuité d'accès.

5.6 Exposition externe via Ingress

L'accès externe est assuré par un Ingress :

Il permet d'exposer le service en dehors du cluster Il gère les règles de routage HTTP/HTTPS Contrairement aux autres composants, il est explicitement indiqué comme non géré par l'Operator Kogito

Cela signifie que :

Sa configuration est laissée à l'utilisateur ou à l'administrateur de la plateforme Il peut être personnalisé selon les besoins (sécurité, certificats TLS, etc.)

5.7 Rôle global de l'Operator (implicite)

Même s'il n'est pas explicitement nommé dans le schéma, l'architecture suggère l'utilisation d'un Operator Kogito :

Automatisation du déploiement Gestion des ressources Kubernetes (pods, services, ConfigMaps) Simplification du cycle de vie applicatif

5.8 Conclusion

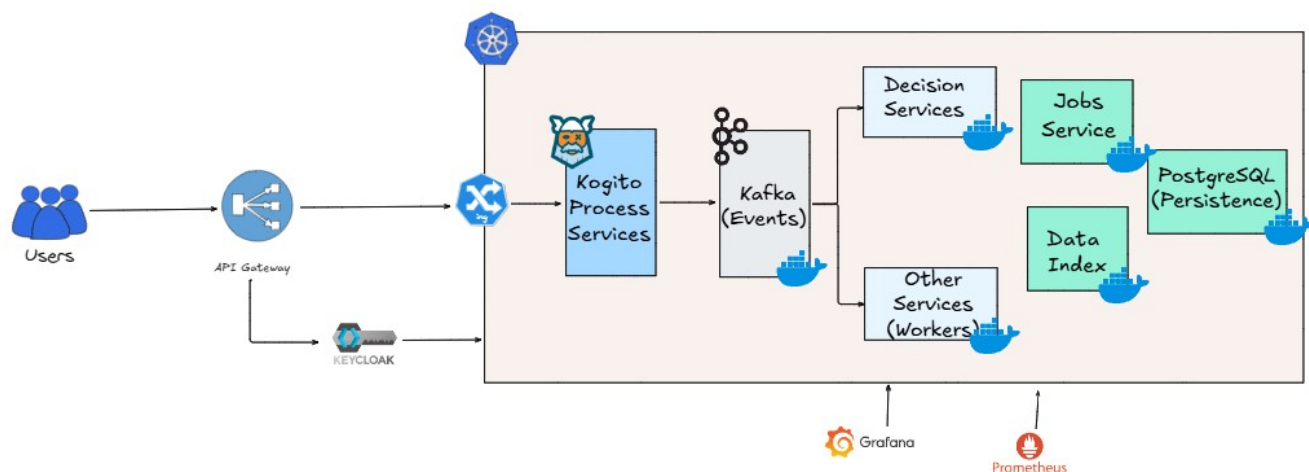
Ce schéma met en évidence une architecture cloud-native typique :

Découplage entre image, configuration et déploiement Scalabilité horizontale via plusieurs pods Configuration dynamique via ConfigMaps Exposition flexible via Service et Ingress

L'ensemble permet de déployer efficacement des services Kogito dans un environnement Kubernetes, tout en assurant robustesse, flexibilité et maintenabilité.

6 Architecture globale du système Kogito

Le schéma illustre une architecture complète de système basé sur Kogito, déployé dans un environnement conteneurisé et orienté événements. Cette architecture met en évidence le flux de traitement depuis l'utilisateur jusqu'aux services métiers, en passant par des composants de sécurité, de streaming de données et de persistance.



6.1 Interaction utilisateur et point d'entrée

Le système est initialement sollicité par des utilisateurs externes qui accèdent aux services via un point d'entrée unique, représenté par un API Gateway. Ce composant joue un rôle central dans la gestion des requêtes entrantes : il assure le routage vers les services appropriés, applique des politiques de sécurité et peut également gérer des aspects tels que le throttling ou la journalisation.

En complément, un mécanisme d'authentification et d'autorisation est assuré par Keycloak, qui permet de sécuriser l'accès aux API. Les utilisateurs doivent s'authentifier au-

près de ce service avant de pouvoir interagir avec les services Kogito, garantissant ainsi un contrôle d'accès robuste basé sur des identités et des rôles.

6.2 Exposition via Ingress et orchestration Kubernetes

Les requêtes passent ensuite par un Ingress Kubernetes, qui permet d'exposer les services à l'extérieur du cluster tout en gérant le routage HTTP/HTTPS. L'ensemble des composants applicatifs est déployé dans un cluster Kubernetes, assurant la gestion des conteneurs, la scalabilité et la résilience.

6.3 Services de processus Kogito

Au cœur de l'architecture se trouvent les services de processus Kogito. Ces services implémentent la logique métier sous forme de workflows (processus BPMN) et orchestrent les différentes étapes de traitement. Ils constituent le point central de coordination entre les différents services du système.

Ces services sont capables de réagir à des événements, d'appeler d'autres services, et de produire de nouveaux événements, s'inscrivant ainsi dans une architecture réactive et découplée.

6.4 Communication événementielle avec Kafka

La communication entre les composants repose sur un bus d'événements implémenté Apache Kafka. Ce système permet de diffuser des événements produits par les services Kogito vers différents consommateurs.

L'utilisation de Kafka présente plusieurs avantages :

- Découplage entre producteurs et consommateurs
- Scalabilité horizontale
- Résilience grâce à la persistance des messages

Les événements générés par les processus métiers sont ainsi consommés par d'autres services spécialisés.

6.5 Services consommateurs et traitement asynchrone

Plusieurs types de services consomment les événements provenant de Kafka :

- Les Decision Services implémentent des règles métier (souvent basées sur DMN) pour prendre des décisions automatisées.
- Le Jobs Service gère les tâches planifiées et les traitements différés.
- Les Other Services (Workers) représentent des services métiers supplémentaires chargés de traitements spécifiques.
- Le composant Data Index est responsable de l'indexation des données générées par les processus afin de faciliter leur consultation et leur recherche.

Cette organisation permet de répartir les responsabilités et d'optimiser les performances globales du système.

6.6 Persistance des données

Les données applicatives et les états des processus sont stockés dans une base de données relationnelle, ici PostgreSQL. Ce composant assure la durabilité des informations et permet la reprise des processus en cas de défaillance.

La persistance est essentielle dans les systèmes Kogito pour garantir la cohérence des workflows et permettre le suivi des instances de processus.

6.7 Supervision et observabilité

L'architecture intègre également des outils de monitoring pour assurer la supervision du système :

Prometheus collecte les métriques des différents services Grafana permet de visualiser ces métriques sous forme de tableaux de bord

Ces outils offrent une visibilité sur les performances, l'état des services et facilitent la détection des anomalies.

6.8 Conclusion

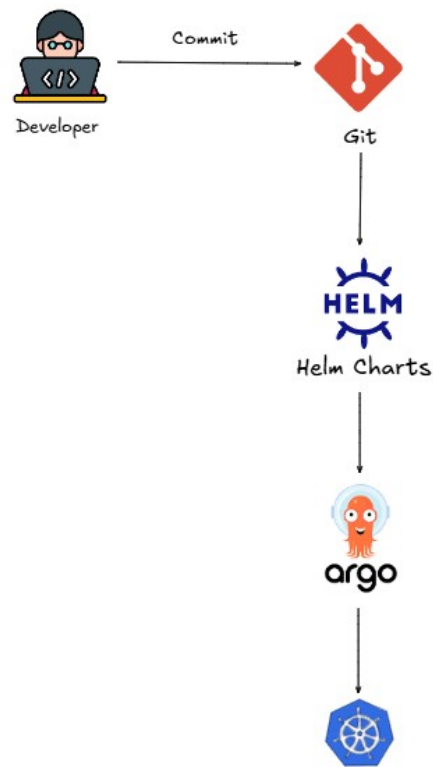
Cette architecture illustre un système moderne basé sur les principes du cloud-native :

Architecture orientée événements avec Kafka Microservices découplés et spécialisés Scalabilité et résilience grâce à Kubernetes Sécurité intégrée avec Keycloak Observabilité complète avec Prometheus et Grafana

Elle permet de concevoir des applications métier robustes, évolutives et capables de traiter des flux de données en temps réel tout en maintenant une forte modularité.

7 Mode de déploiement

Choix retenu : Helm + GitOps



Helm

- packaging des applications
- versionning
- **GitOps (ArgoCD)**
- déploiement automatique
- rollback facile

Conclusion générale